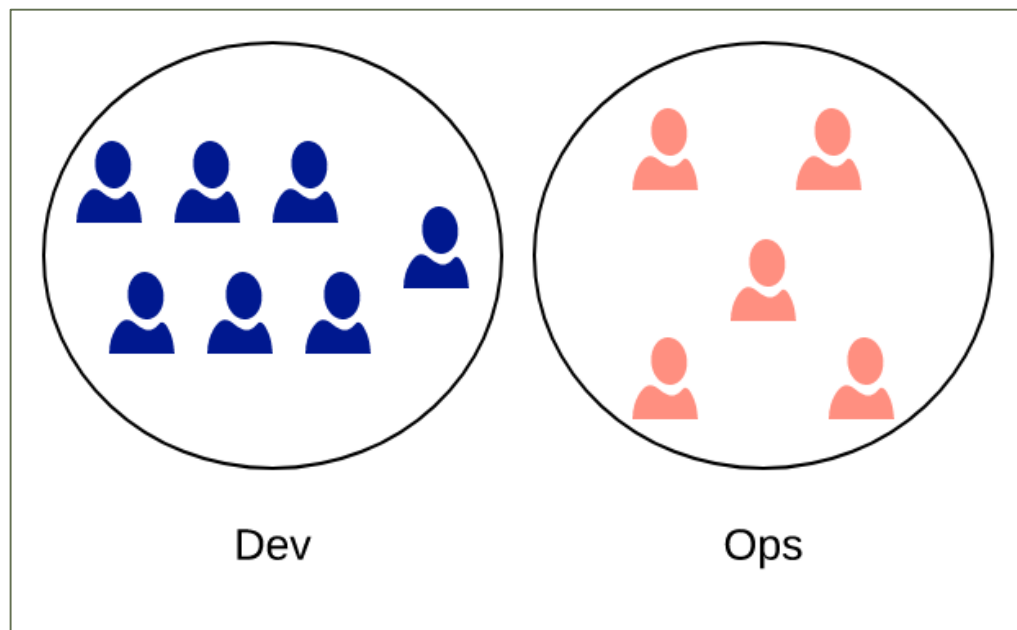


DevOps

UNIFICANDO AS CULTURAS DE DESENVOLVIMENTO (DEV)
E DE OPERAÇÃO (OPS)

A "última milha " de um ciclo de produção de
software:
implantação do sistema

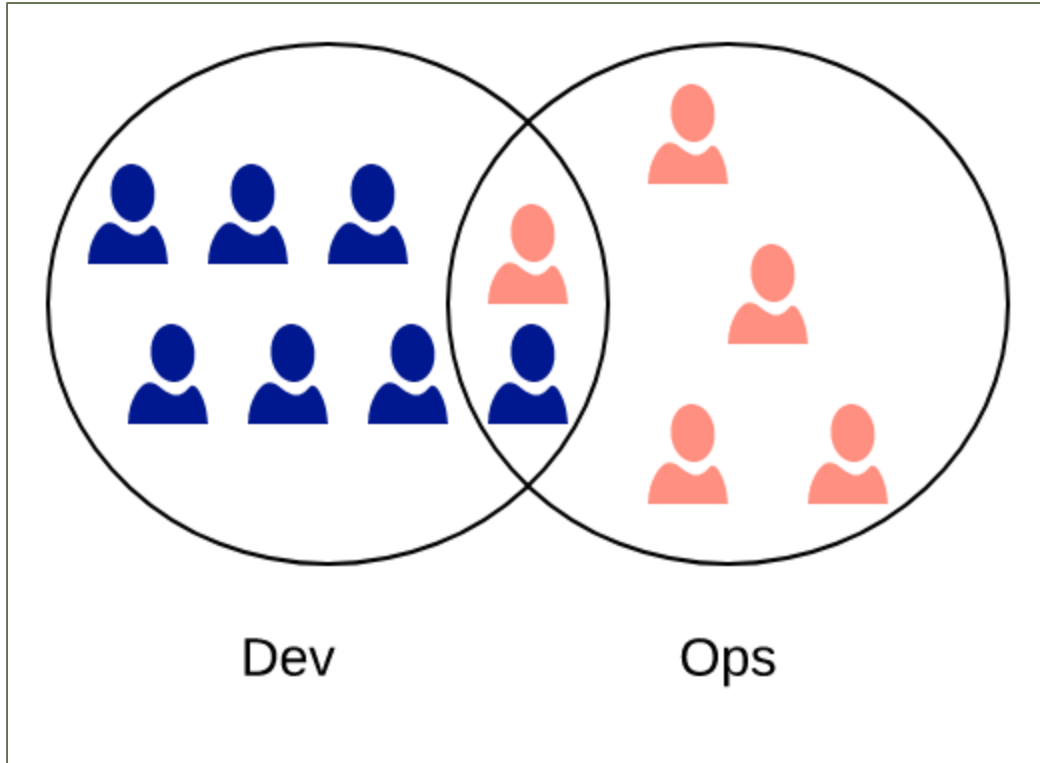
Implantação pode ser traumática



Dois nichos independentes,
com pouquíssima comunicação

Ops = administradores de
sistema, suporte, sysadmin,
pessoal de IT, etc

Ideia central de DevOps: aproximar Dev e Ops



Objetivo: passagem de bastão bem sucedida!

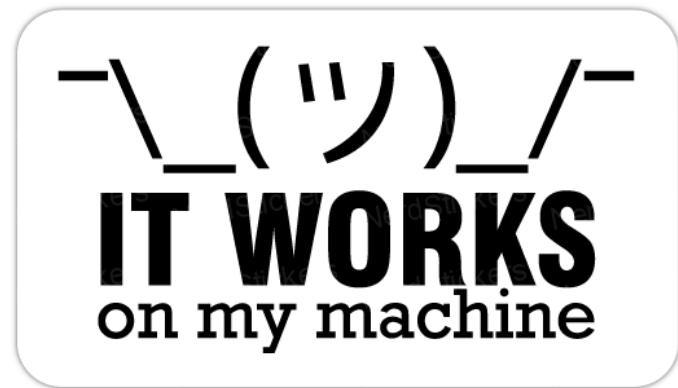
(Deve começar o quanto antes; ser automatizada sempre que possível)

Dev



Ops

Objetivo: acabar com o empurra-empurra



Dev: o problema não é meu código, mas seu servidor

Ops: o problema não é meu servidor, mas o seu código

DevOps

- Não é um cargo ou método, mas um conjunto de princípios e práticas
- Termo surgiu em 2009

Princípios de DevOps

- Aproximar Devs e Ops, desde o início do projeto
- Adotar princípios ágeis na fase de implantação
- Transformar implantações em um não-evento
- Implantar sistemas, ou partes dele, todos os dias
- Automação do processo de implantação

Práticas de DevOps

- Controle de Versões
- Integração Contínua
- Deployment Contínuo

Controle de Versões

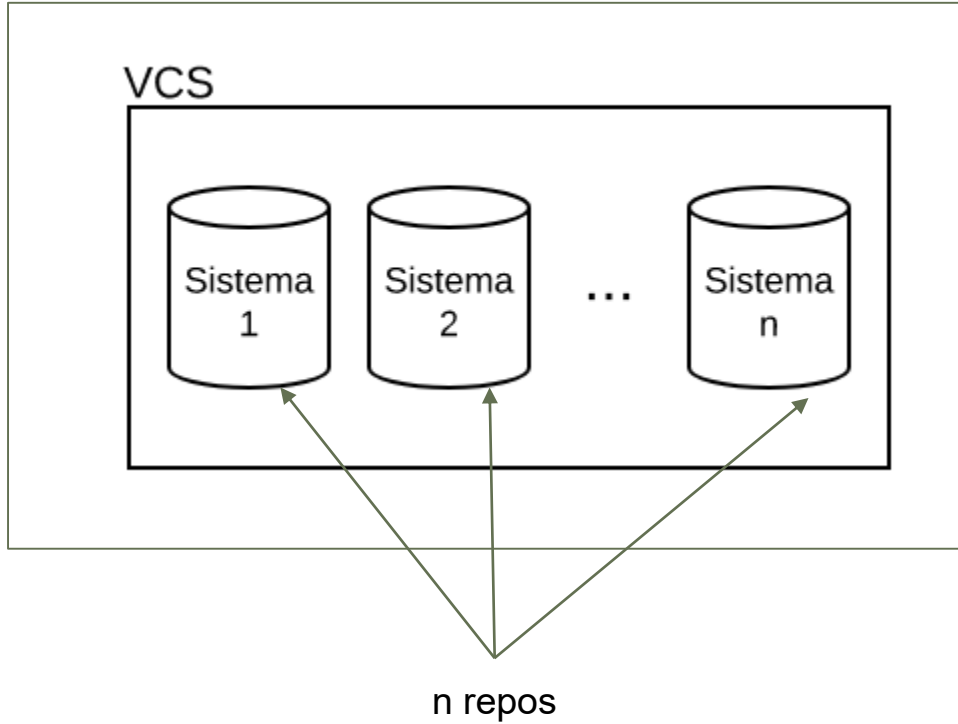
Controle de Versões

Version Control System (VCS)

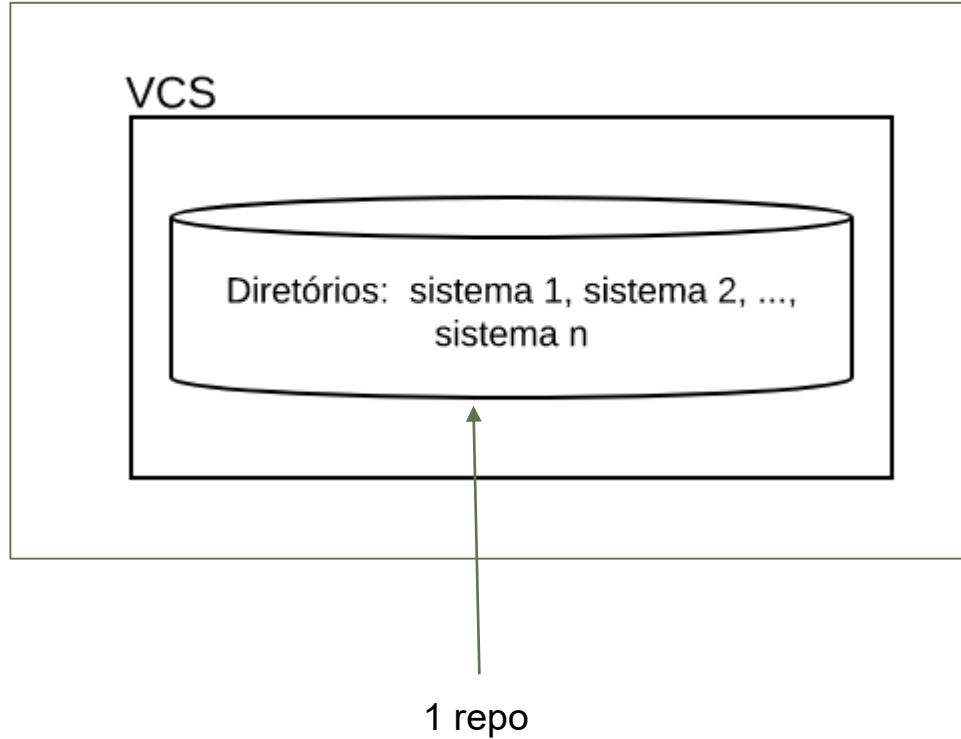
- Fundamental para desenvolvimento colaborativo
- Armazena "última" versão do sistema (fonte da verdade)
- Permite recuperar versões anteriores

Multirepositório vs monorepositório

Multirepositório (mais comum)



Monorepositório (um pouco menos comum)



DOI:10.1145/2854146

Google's monolithic repository provides a common source of truth for tens of thousands of developers around the world.

BY RACHEL POTVIN AND JOSH LEVENBERG

Why Google Stores Billions of Lines of Code in a Single Repository

This article outlines the scale of that codebase and details Google's custom-built monolithic source repository and the reasons the model was chosen. Google uses a homegrown version-control system to host one large codebase visible to, and used by, most of the software developers in the company. This centralized system is the foundation of many of Google's developer workflows. Here, we provide background on the systems and workflows that make feasible managing and working productively with such a large repository. We explain Google's "trunk-based development" strategy and the support systems that structure workflow and keep Google's codebase healthy, including software for static analysis, code cleanup, and streamlined code review.

Google-Scale

Google's monolithic software repository, which is used by 95% of its software developers worldwide, meets the definition of an ultra-large-scale^a system, providing evidence the single-source repository model can be scaled successfully.

The Google codebase includes approximately one billion files and has a history of approximately 35 million commits spanning Google's entire 18-year existence. The repository contains 86TB^a of data, including approximately

^a Total size of uncompressed content, excluding release branches.

Monorepositórios são usados principalmente por grandes empresas

<https://research.google/pubs/pub45424/>

Vantagens de Monorepositórios

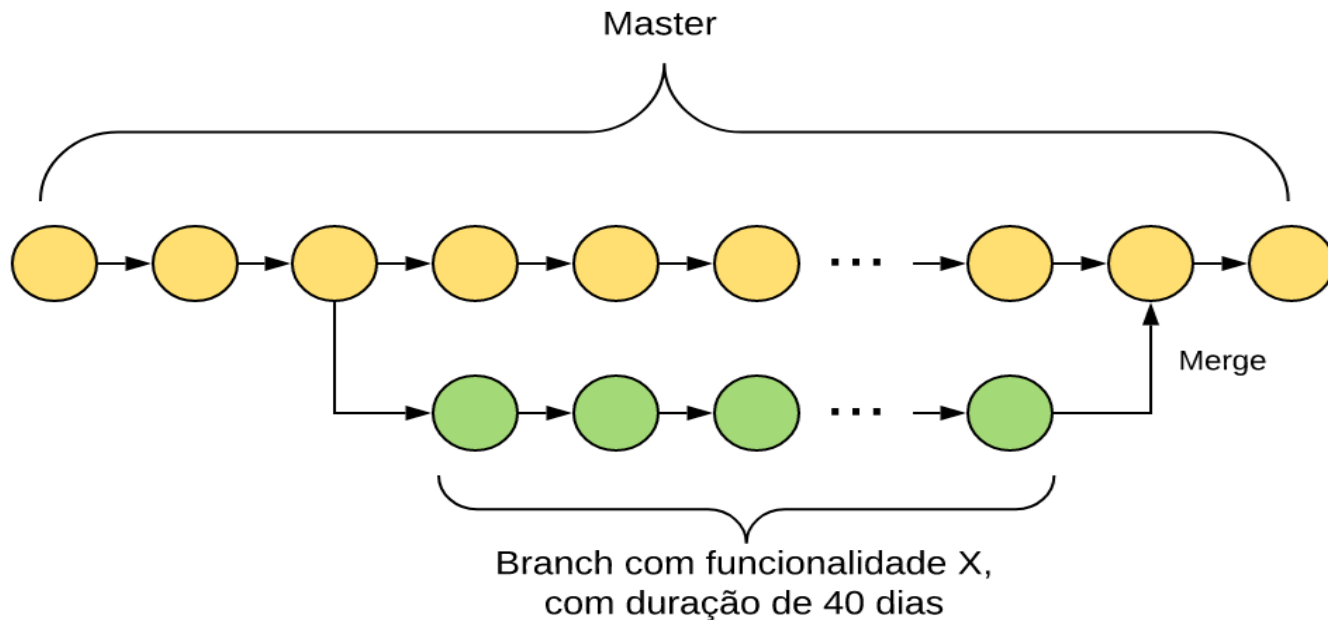
- Uma única fonte de "verdade"
- Incentivam reúso de código
- Mudanças são atômicas (1 commit pode alterar n sistemas)
- Facilitam a refatoração em larga escala

Desvantagem de Monorepositório

- Cada desenvolvedor terá em seu repositório local todos os arquivos de todos os sistemas da organização
 - Requer ferramentas específicas para navegar em grandes bases de código.

Integração Contínua

Branches longos eram comuns



Se uma tarefa causa "dor", o melhor é não deixar ela
acumular e fazer um pouco todo dia

A solid orange horizontal bar at the bottom of the slide.

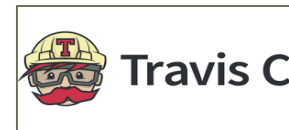
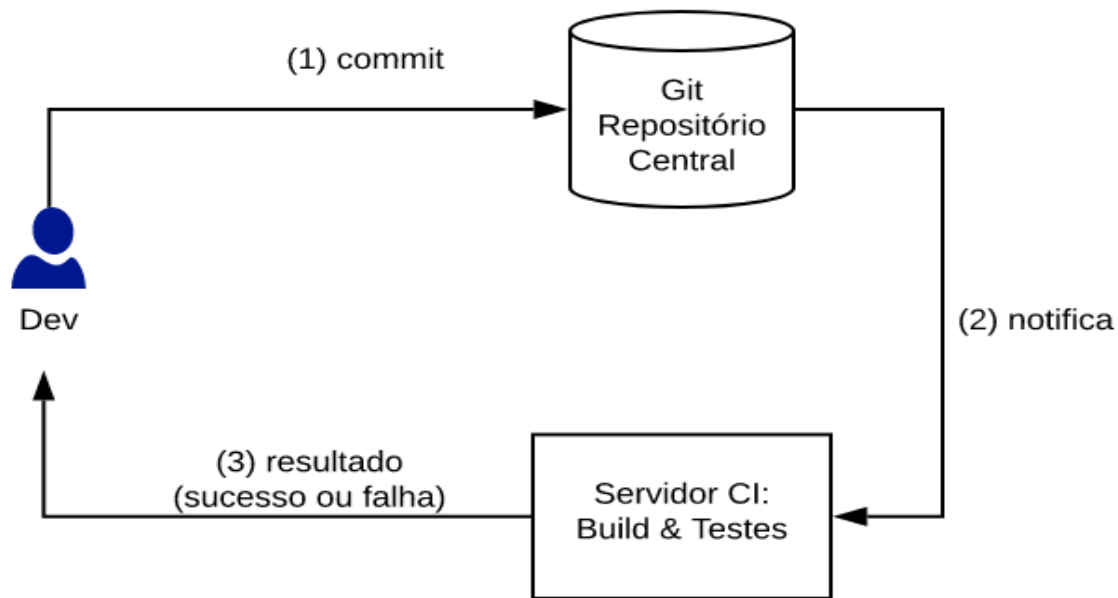
Integração Contínua (CI)

- Prática proposta por XP
- Como o nome diz, integrar o código com frequência
- Mas com qual frequência?
 - Não existe consenso, mas a maioria dos autores recomenda pelo menos 1 vez por dia

Boas práticas com CI

- Build automatizado
- Testes automatizados
- Programação em pares

Servidores de CI



Desenvolvimento baseado no Branch Principal

(Trunk Based Development - TBD)

- Como merges podem gerar conflitos, TBD defende:
 - Não usar mais branches de funcionalidades
 - Implementá-las diretamente no branch principal
- Branch principal = trunk, master



Quase todo desenvolvimento ocorre no HEAD do repositório. Isso ajuda a identificar problemas de integração mais cedo e minimiza o esforço para realização de merges.



Todos engenheiros trabalham em um único branch... o que torna o desenvolvimento mais rápido, pois não dispende-se esforço na integração de branches de longa duração no trunk.

Deployment Contínuo

Deployment Contínuo (CD)

- Próximo passo:
 - Commits/merge são realizados com frequência (CI)
 - E entram imediatamente em produção (CD)
- Objetivo: experimentação e feedback!

The Netflix logo, consisting of the word "NETFLIX" in a bold, red, sans-serif font, is centered within a thin black rectangular border.

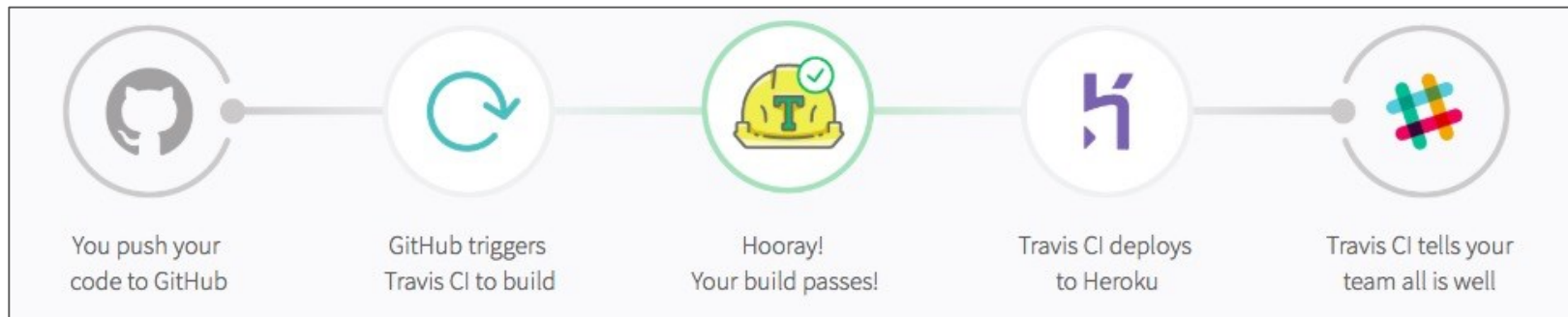
Se não houver usuários "passando o mouse" sobre um novo elemento da UI, um experimento pode mover esse elemento para um novo local na tela.

Se todos os experimentos mostrarem falta de interesse, o elemento será removido do produto

Fonte: The Top 10 Adages in Continuous Deployment, IEEE Software 2017

A solid orange gradient bar at the bottom of the slide, transitioning from a lighter shade on the left to a darker shade on the right.

Possível workflow para Deployment Contínuo



História da Engenharia de Software:

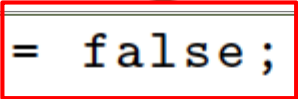
encurtamento de ciclos

	Desenvolvimento	Entrada em Produção
Waterfall	meses (ou anos)	meses (ou anos)
Ágil	semanas	meses
Ágil + CD	semanas	imediatamente

Como evitar que minhas implementações parciais
cheguem até os usuários finais?

Feature Flags/Toggles

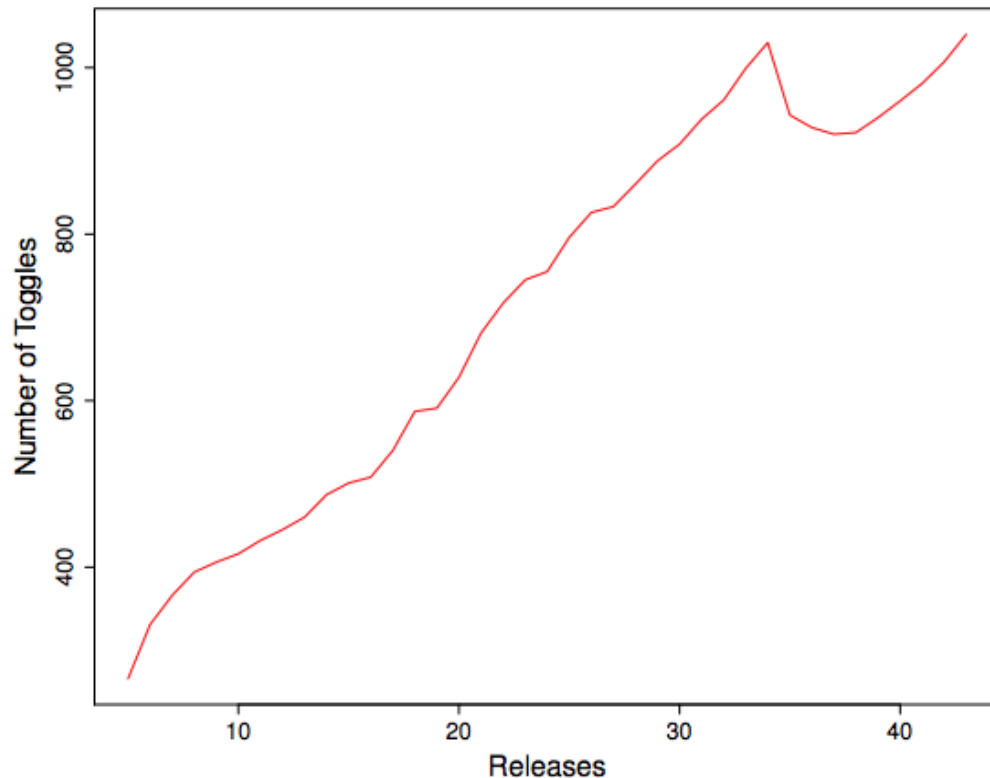
Enquanto a feature estiver em desenvolvimento!



```
featureX = false;  
...  
if (featureX)  
    "aqui tem código incompleto de X"  
...  
if (featureX)  
    "mais código incompleto de X"
```


Quando código ficar pronto: habilita-se flag

```
featureX = true;  
...  
if (featureX)  
    "aqui tem código incompleto de X"  
...  
if (featureX)  
    "mais código incompleto de X"
```



Md Tajmilur Rahman et al. Feature toggles: practitioner practices and a case study. MSR 2016.

Figure 2: Number of unique toggles per release of Google Chrome.

Referências

- [1] Gene Kim, Jez Humble, John Willis, Patrick Debois. Manual de DevOps. Como Obter Agilidade, Confiabilidade e Segurança em Organizações Tecnológicas. Alta Books, 2018.
- [2] Jez Humble, David Farley. Entrega Contínua: Como Entregar Software de Forma Rápida e Confiável. Bookman, 2014.
- [3] Steve Matyas, Andrew Glover, Paul Duvall. Continuous Integration: Improving Software Quality and Reducing Risk. Addison-Wesley, 2007.
- [4] Valente, M. T. Engenharia de Software Moderna. Editora Independente, 2020.